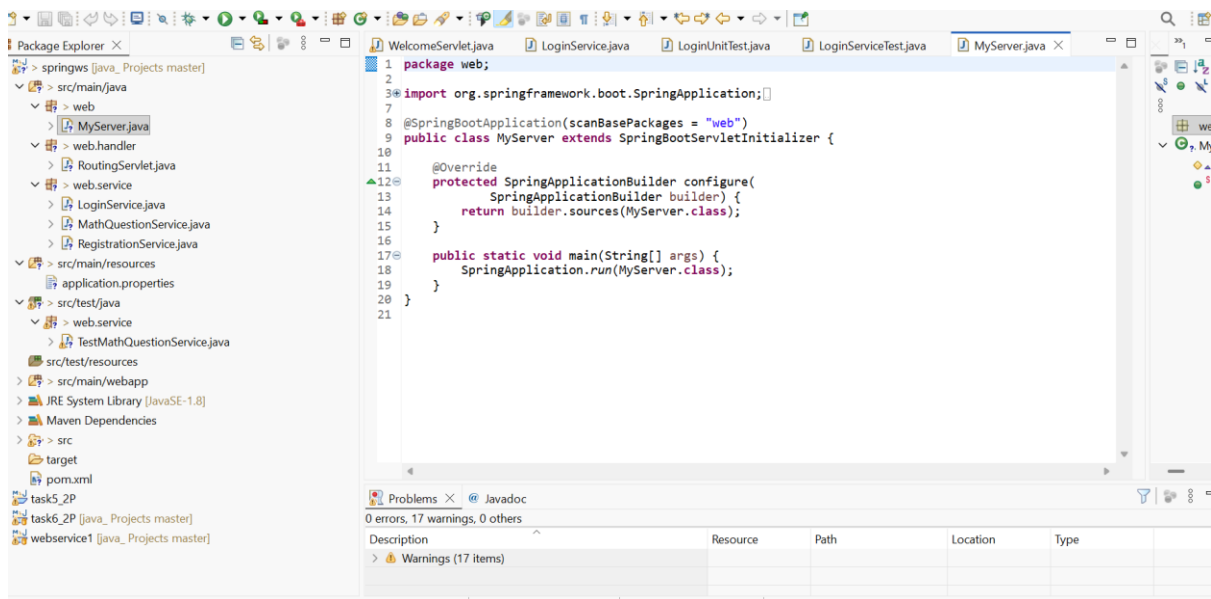
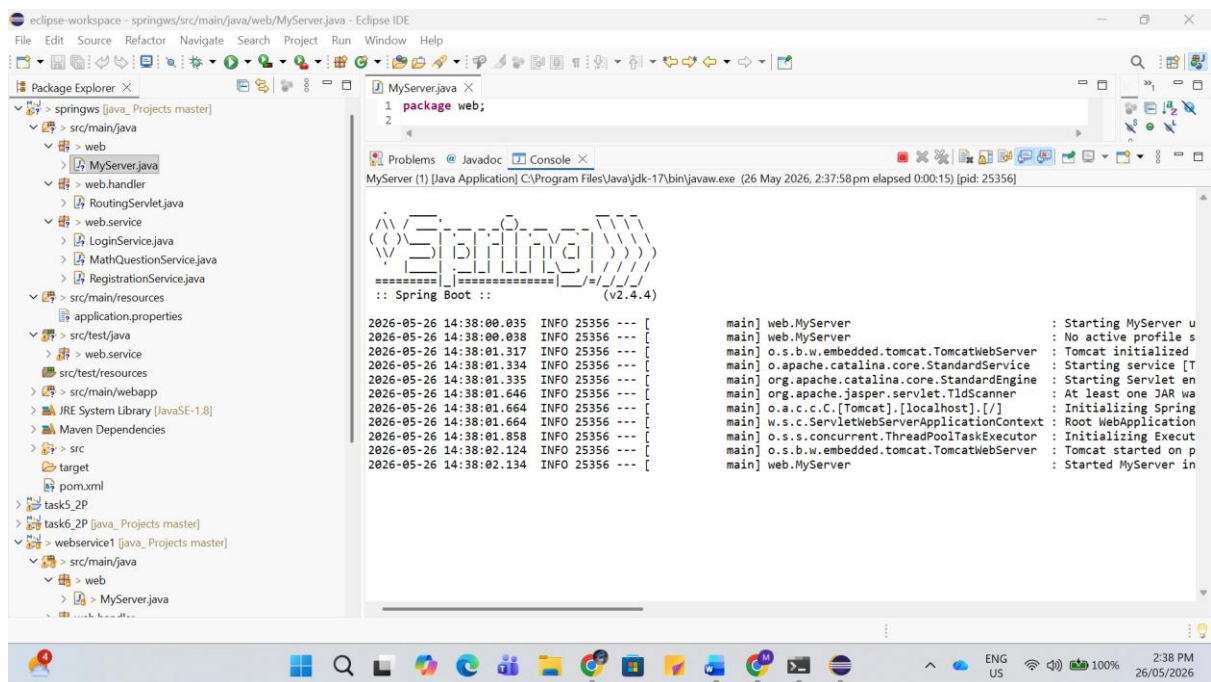


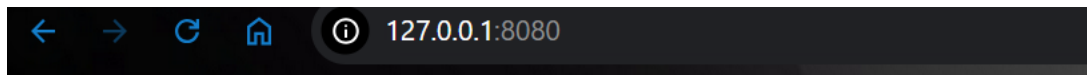
Pass Task 8: Turn 7.1P into a STEM game for grade 4-6 kids\



Import Successfully 8.1p springws



Ran Myserver



Welcome

Server is running...

Go to [Login](#)



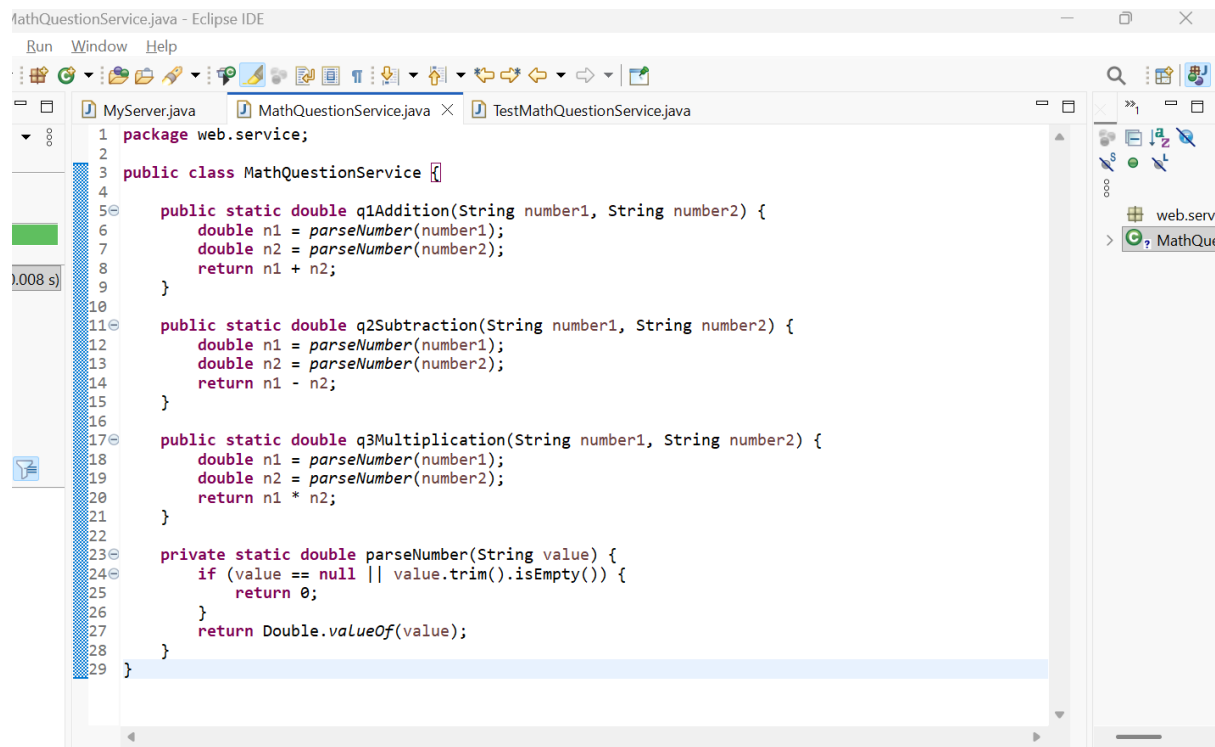
HTML Forms

User name:

Password:

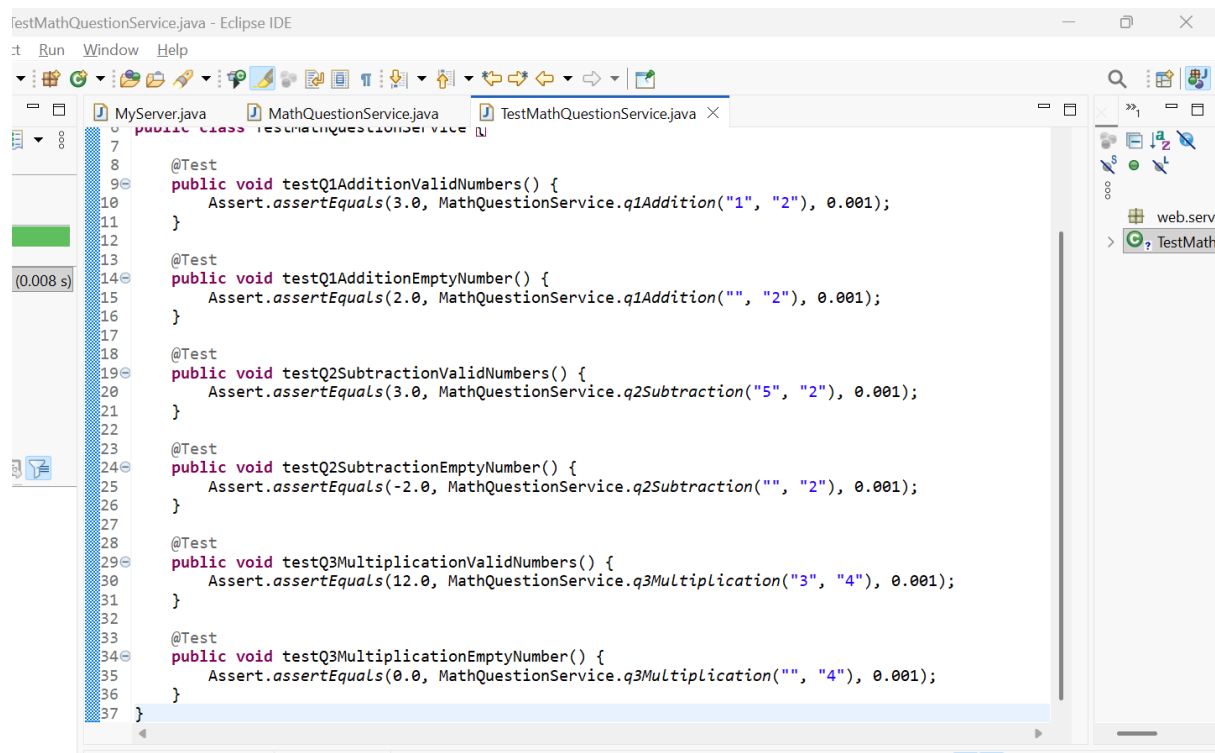
DoB:
 

If you click the "Login" button, the form-data will be sent to login-servlet.



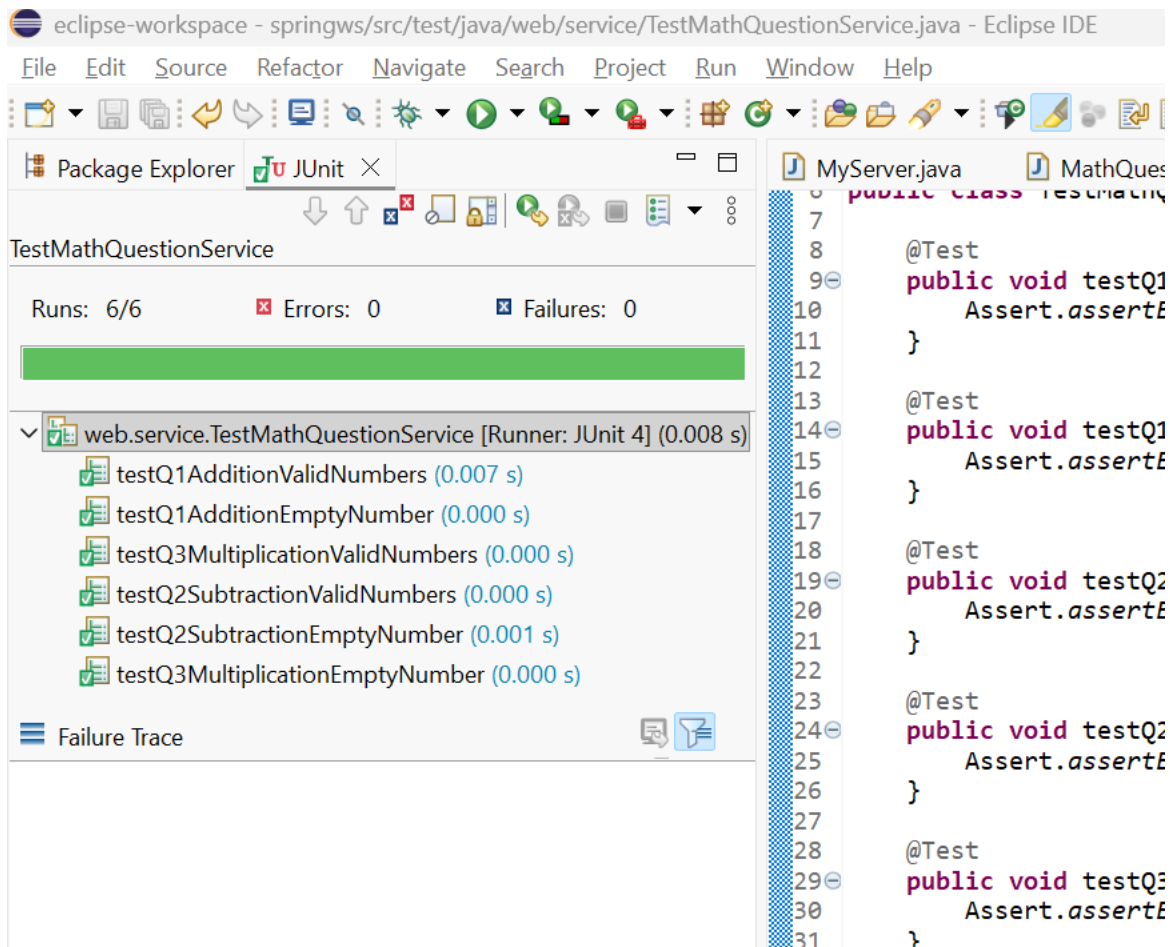
```
1 package web.service;
2
3 public class MathQuestionService {
4
5     public static double q1Addition(String number1, String number2) {
6         double n1 = parseNumber(number1);
7         double n2 = parseNumber(number2);
8         return n1 + n2;
9     }
10
11     public static double q2Subtraction(String number1, String number2) {
12         double n1 = parseNumber(number1);
13         double n2 = parseNumber(number2);
14         return n1 - n2;
15     }
16
17     public static double q3Multiplication(String number1, String number2) {
18         double n1 = parseNumber(number1);
19         double n2 = parseNumber(number2);
20         return n1 * n2;
21     }
22
23     private static double parseNumber(String value) {
24         if (value == null || value.trim().isEmpty()) {
25             return 0;
26         }
27         return Double.valueOf(value);
28     }
29 }
```

Updated the mathquestion service file



```
1 public class TestMathQuestionService {
2
3     @Test
4     public void testQ1AdditionValidNumbers() {
5         Assert.assertEquals(3.0, MathQuestionService.q1Addition("1", "2"), 0.001);
6     }
7
8     @Test
9     public void testQ1AdditionEmptyNumber() {
10         Assert.assertEquals(2.0, MathQuestionService.q1Addition("", "2"), 0.001);
11     }
12
13     @Test
14     public void testQ2SubtractionValidNumbers() {
15         Assert.assertEquals(3.0, MathQuestionService.q2Subtraction("5", "2"), 0.001);
16     }
17
18     @Test
19     public void testQ2SubtractionEmptyNumber() {
20         Assert.assertEquals(-2.0, MathQuestionService.q2Subtraction("", "2"), 0.001);
21     }
22
23     @Test
24     public void testQ3MultiplicationValidNumbers() {
25         Assert.assertEquals(12.0, MathQuestionService.q3Multiplication("3", "4"), 0.001);
26     }
27
28     @Test
29     public void testQ3MultiplicationEmptyNumber() {
30         Assert.assertEquals(0.0, MathQuestionService.q3Multiplication("", "4"), 0.001);
31     }
32 }
33 }
```

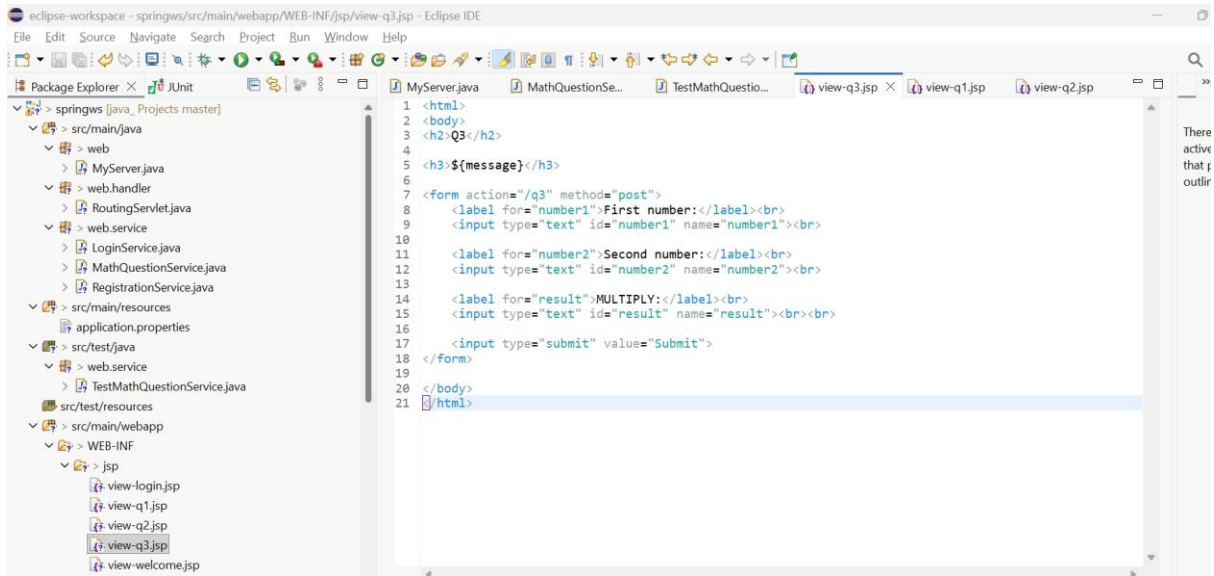
Updated the testmathquestionservice file



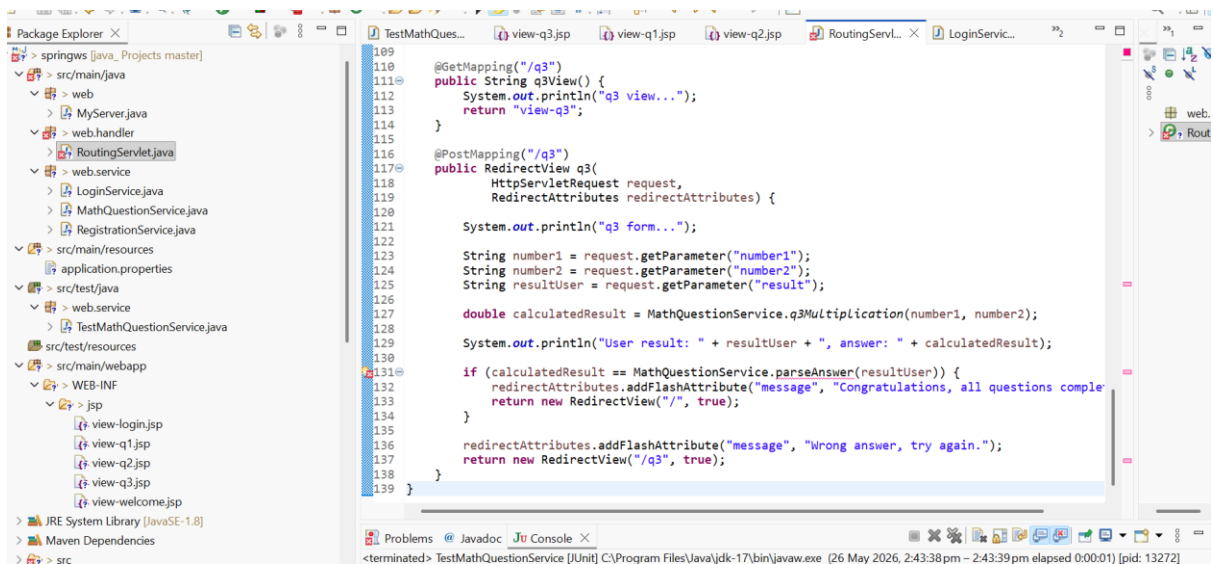
Runs perfectly.

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
springws	23.7 %	87	280	
src/main/java	13.0 %	42	280	
web.handler	0.0 %	0	203	
web.service	41.6 %	42	59	
web	0.0 %	0	18	
src/test/java	100.0 %	45	0	
web.service	100.0 %	45	0	
TestMathQuestionService.java	100.0 %	45	0	

Created q3



Updated the RoutingServlet



After logging in

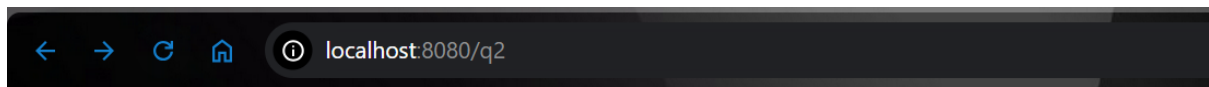


Q1

First number:

Second number:

ADD:

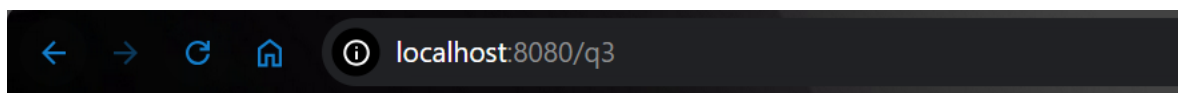


Q2

First number:

Second number:

Subtraction:



Q3

First number:

Second number:

MULTIPLY:

Wrong Answer stops you from going forward

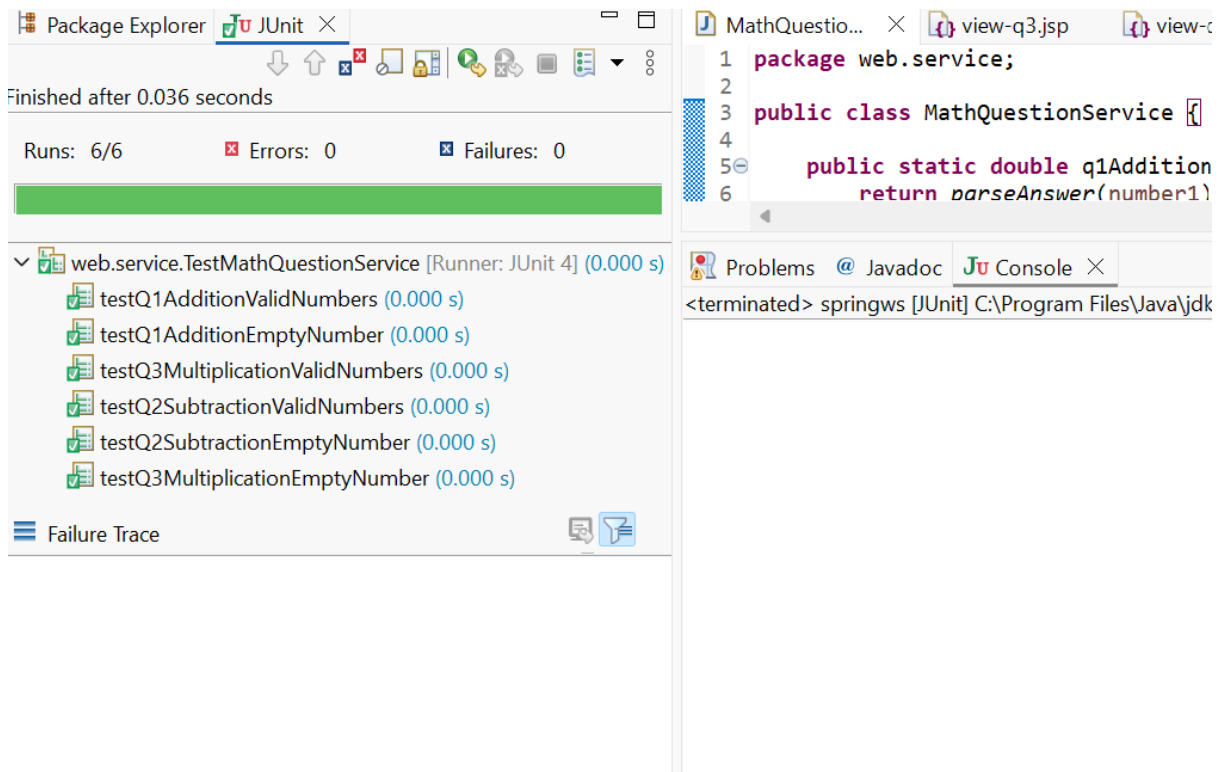
Q1

Wrong answer, try again.

First number:

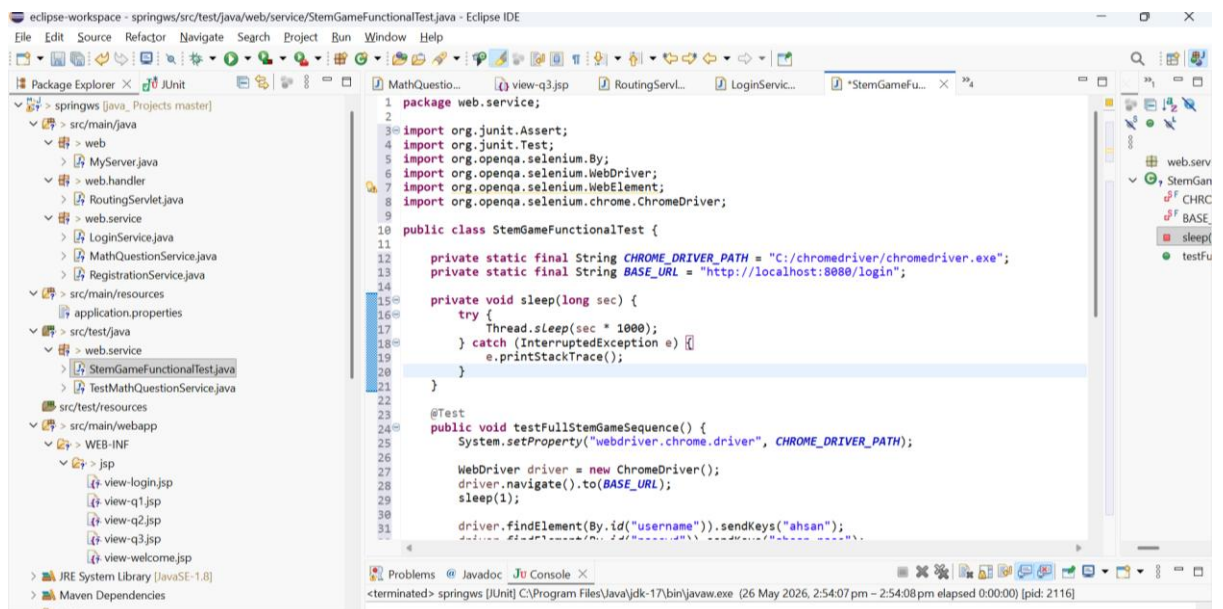
Second number:

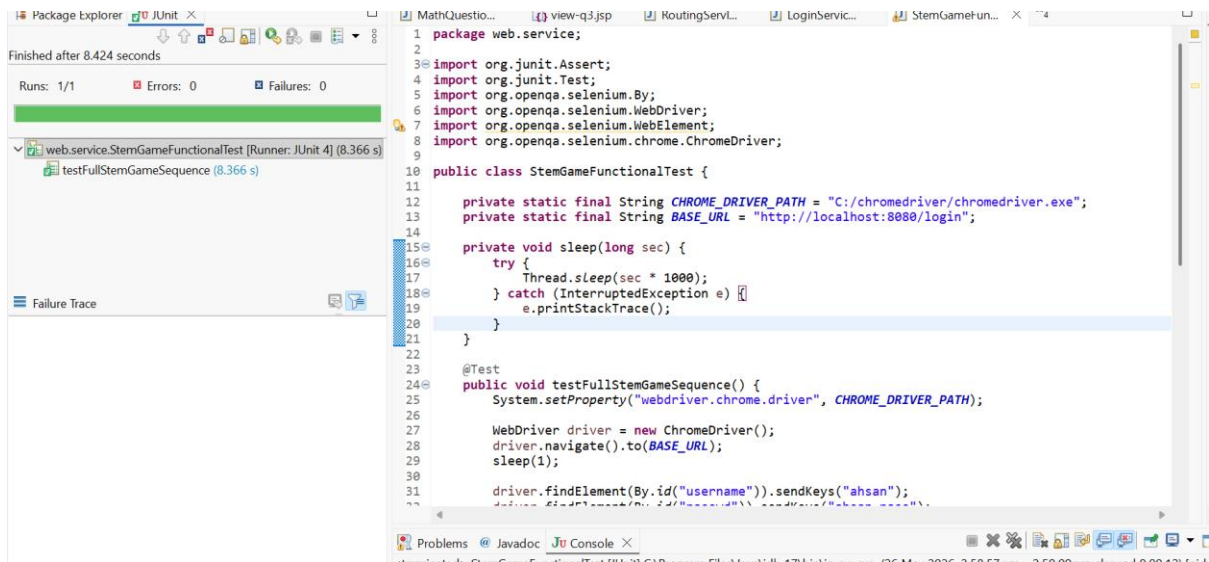
ADD:



Everything runs fine and checks junit test

Added a stemgamefunctionaltest





Self-Reflection on Testing an Integrated Web Application Compared to 7.1P

Compared to Task 7.1P, this task was more complex because it involved testing a complete integrated web application instead of testing only standalone functions. In 7.1P, the focus was mainly on unit testing individual Java methods using JUnit and checking code coverage. However, in this task, both backend Java logic and frontend web pages had to work together correctly. I learned how routing, JSP pages, form submission, and service classes interact within a Spring web application.

One major challenge was handling invalid or empty input values entered through the web pages. Initially, empty values caused the application to display an error page. By improving the MathQuestionService functions and adding unit tests, the system became more stable and user-friendly. I also learned how Selenium can automate browser interaction and simulate real user behaviour during functional testing.

This task improved my understanding of integrated testing because it required testing the complete workflow from login to question pages instead of testing isolated functions only. It also demonstrated the importance of validating both success and failure scenarios in a web application environment.

Reflection on Functional Testing Sequence of Pages Using JUnit and Best Testing Practices

Functional testing using Selenium and JUnit helped verify whether the web application pages behaved correctly from a user's perspective. The testing sequence included login validation, navigation between Q1, Q2, and Q3 pages, correct answer handling, and incorrect answer handling. By automating the page flow, it became easier to repeatedly test the application without manually entering values each time.

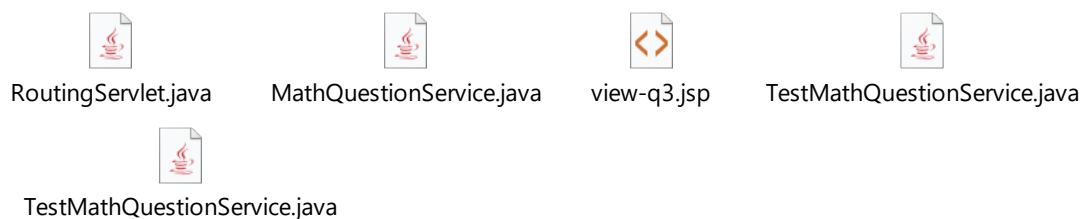
JUnit was useful for organising test cases into separate methods with clear purposes. Unit tests were separated from Selenium functional tests to improve readability and maintainability. Naming test methods clearly, such as `testLoginSuccess()` and `testQ1AdditionValidNumbers()`, made the testing process easier to understand and debug.

Best testing practices followed during this task included:

- Testing both valid and invalid inputs.
- Separating unit tests from functional tests.
- Using assertions to verify expected behaviour.
- Keeping test methods small and focused on one purpose.
- Re-running tests after each major code update.

Overall, the task highlighted how structured testing improves software reliability and helps identify issues early during development.

Source code



Github Screenshot

